Unity Merge Integration – Preliminary Inspection Report

1. Scope

A read-only inspection was performed on the merged Unity project before starting the planned work on:

- ETA System;
- Internal Screen System;
- Safety Interaction System.

The analysis covered C# scripts, Unity prefabs, scene serialization, layers, physics configuration, runtime instantiation, event wiring, and system dependencies.

No project files or Unity assets were modified during this inspection.

---

2. Executive Summary

The core project architecture is present and the player vehicle is instantiated from the correct `TestCar.prefab`.

The Safety System is also physically present inside the player-car prefab, including:

- detection trigger;
- Rigidbody;
- audio source;
- warning lights;
- safety state component.

However, several integration problems currently prevent the ETA, Internal Screen, and Safety systems from working reliably.

The most important blockers are:

1. NPC pedestrians cannot currently trigger the Safety System because their prefabs have no Collider or Rigidbody.
2. Two Unity layers are named `DynamicUser`, creating inconsistent layer assignments.
3. The Safety detector monitors Layer 3, while NPC pedestrians are assigned to Layer 6.
4. The internal safety overlay is resolved through a global hardcoded `GameObject.Find` path, but the target object is inactive and therefore cannot be found.
5. The Totem recall flow contains multiple competing entry points and can calculate the ETA for the wrong vehicle.
6. Safety audio and light dependencies are assigned through reflection at runtime.

7. Safety adapters are duplicated dynamically even though equivalent components already exist in the prefab.
8. The current Safety stop logic bypasses smooth braking and immediately stops the NavMeshAgent.
9. The ETA system has a null `ParkingTrafficProvider` reference, so traffic delays are currently ignored.
10. The standalone Safety prefab and the copy embedded in `TestCar.prefab` are no longer linked.

These issues should be resolved before implementing the final Figma interfaces and performing full-system validation.

---

3. Confirmed Correct Elements

Player-car instantiation

`PlayerCarProgression` references the correct `TestCar.prefab` and instantiates it during runtime.

Safety System presence

The following hierarchy exists inside `TestCar.prefab`:

`TestCar/AHMI_SafetyInteractionSystem`

The embedded Safety System contains:

- a `DetectionZone`;
- a trigger `SphereCollider`;
- a `SafetyTriggerDetector`;
- a kinematic Rigidbody;
- gravity disabled.

Safety audio and warning lights

The following components are present:

- an AudioSource under the Safety Audio hierarchy;
- left and right warning lights;
- Safety audio and light adapters.

Runtime safety subscription

`SmartCarNavigator.Start()` locates the `SafetyInteractionState` and subscribes programmatically to:

- `OnSafetyWaitStarted`;
- `OnSafetyWaitEnded`.

Card reader debounce

`CardReader.cs` includes a 1.5-second cooldown, preventing repeated card reads within a very short interval.

---

4. Critical Blockers

4.1 NPC pedestrians have no physics components

The inspected NPC pedestrian prefabs contain neither a Collider nor a Rigidbody.

Runtime consequence

The car Safety trigger cannot receive `OnTriggerEnter` or `OnTriggerExit` callbacks when a pedestrian crosses the detection zone.

Required correction

Add an appropriate Collider, normally a CapsuleCollider, to the relevant pedestrian root or physics object.

A Rigidbody should only be added if required by the final movement architecture. Since the car Safety zone already contains a Rigidbody, a pedestrian Collider may be sufficient, but this must be verified in Play Mode.

4.2 Duplicate `DynamicUser` layer names

The Unity project currently contains two different layer indexes with the same name:

- Layer 3: `DynamicUser`;
- Layer 6: `DynamicUser`.

Runtime consequence

Scripts, LayerMasks, and prefabs may appear correctly configured by name while actually referencing different numeric layers.

Required correction

Select one authoritative `DynamicUser` layer index and migrate every relevant object and LayerMask to that index.

The duplicate layer name should then be removed or renamed.

4.3 Safety LayerMask mismatch

The `SafetyTriggerDetector` in `TestCar.prefab` currently monitors Layer 3.

NPC pedestrians are assigned to Layer 6.

Runtime consequence

Even after adding Colliders, pedestrians will still not be detected.

Required correction

After selecting the authoritative layer, update:

- NPC pedestrian prefab layers;
- vehicle and obstacle layers;
- `SafetyTriggerDetector.detectableLayers`;
- any equivalent LayerMasks in `SmartCarNavigator`;
- the Physics collision matrix.

4.4 Broken internal-screen hazard overlay resolution

The Safety screen adapter searches for the following global hierarchy:

`AHMI_InternalScreenSystem/tesla/HUD/HazardOverlay`

The search uses `GameObject.Find`.

The `HazardOverlay` object is inactive by default.

Runtime consequence

`GameObject.Find` does not return inactive GameObjects. Therefore, the lookup returns `null`, and the safety overlay does not appear.

The path is also fragile because it depends on:

- the exact root name;
- the exact child name `tesla`;
- a fixed hierarchy;
- only one matching screen in the scene.

Required correction

Resolve the overlay relative to the instantiated player car, or assign the reference directly in the prefab.

The adapter should not rely on a global scene search.

4.5 Totem recall and ETA ambiguity

The card-tap recall flow currently invokes both:

- the authoritative car recall through `ValetGuidanceSystem`;
- an additional `TotemCarRequest.RequestCar()` call.

The keyboard fallback using `E` also calls `RequestCar()` independently.

`RequestCar()` searches globally for a `CarETAOrchestrator`, which may return the orchestrator belonging to an unrelated car.

Runtime consequence

Possible effects include:

- duplicated route setup;
- duplicated ETA calculations;
- wrong ETA displayed;
- wrong car selected;
- billboard updates not matching the recalled vehicle;
- `E` updating the ETA without physically recalling the car.

Required correction

`ValetGuidanceSystem` should remain the authoritative recall controller.

The recalled session and its car-specific `CarETAOrchestrator` should be passed explicitly through the recall flow.

Global searches for a random `CarETAOrchestrator` should be removed.

4.6 ETA traffic provider reference is null

The `parkingTrafficProvider` field in `CarETAOrchestrator` is null inside `TestCar.prefab`.

No reliable fallback currently resolves the scene-level `ParkingTrafficProvider`.

Runtime consequence

Traffic-delay contribution evaluates to zero.

The ETA may update, but it does not reflect the actual number of active vehicles.

Required correction

Connect the runtime vehicle orchestrator to the authoritative scene-level `ParkingTrafficProvider`.

The preferred architecture is explicit dependency injection from the central traffic or valet manager rather than an unrestricted global scene search.

---

5. High-Risk Integration Issues

5.1 Runtime reflection

`SmartCarNavigator.SetupSafetyAudioAndLights()` uses reflection to assign private adapter fields, including:

- `audioSource`;
- `lights`.

Risks

- violation of the project rules in `AI_CONTEXT.md`;
- fragility after private-field renaming;
- dependency on implementation details;
- potential IL2CPP/AOT stripping issues;
- difficult debugging.

Recommended correction

Use serialized references, explicit initialization methods, or strongly typed dependency injection.

5.2 Duplicate Safety adapters

`SmartCarNavigator.Start()` dynamically adds:

- `SafetyAudioAdapter`;
- `BlinkingLightAdapter`.

Equivalent components already exist inside the Safety hierarchy of `TestCar.prefab`.

Runtime consequence

The spawned player car may contain two instances of each adapter:

- one inside the Safety System;
- one dynamically added to the root.

This can cause duplicated audio, duplicated light updates, conflicting states, and unclear ownership.

Recommended correction

Use the existing prefab components as the single authoritative adapters.

Avoid runtime `AddComponent` when the component is already part of the prefab.

5.3 Immediate stop conflicts with smooth braking

When `isSafetyWaiting` becomes true, `SmartCarNavigator` currently forces:

- `agent.isStopped = true`;
- `currentSpeed = 0`.

The project also contains smooth-braking adapters.

Runtime consequence

The immediate stop overrides the gradual deceleration logic.

The vehicle therefore stops abruptly instead of performing the intended smooth autonomous braking.

Recommended correction

Define one authoritative owner of vehicle speed during a Safety event.

The Safety state should request braking, while the selected braking adapter gradually reduces speed and only sets the agent to a full stopped state once the speed reaches zero.

5.4 Broken serialized Safety events

Serialized event entries in the Safety prefab and `TestCar.prefab` contain empty method names or invalid targets.

The system currently works only because `SmartCarNavigator` subscribes programmatically at runtime.

Runtime consequence

The Inspector wiring is misleading and cannot be relied upon.

Changes to the runtime subscription logic may silently disable parts of the system.

Recommended correction

Choose one event-wiring model:

- explicit programmatic subscriptions; or
- valid serialized UnityEvents.

Avoid maintaining a broken combination of both.

5.5 Unpacked Safety prefab

The Safety System embedded in `TestCar.prefab` is an unpacked copy rather than a nested instance of:

`AHMI_SafetyInteractionSystem.prefab`

Runtime consequence

Changes made to the standalone Safety prefab do not propagate to the player-car prefab.

Recommended correction

Decide which asset is authoritative.

The preferred solution is to restore a nested prefab relationship, provided doing so does not overwrite intentional car-specific overrides.

5.6 Runtime initialization timing

`SmartCarNavigator` is dynamically added through `PlayerCarProgression`.

Its `Start()` method therefore initializes the Safety dependencies on the following frame.

Runtime consequence

There is a short period in which navigation may start before all Safety adapters and subscriptions are initialized.

Recommended correction

Configure the navigator before vehicle movement begins or provide an explicit initialization sequence.

---

6. Secondary Issues

Wheel reference names

`SmartCarNavigator` searches for wheel objects named:

- `Wheel_FL`;
- `Wheel_FR`;
- `Wheel_RL`;
- `Wheel_RR`.

The player-car prefab uses different names.

Consequence

Wheel spin and steering animations do not work.

This issue is visual and can be deferred.

Parking spot list

`ValetGuidanceSystem.allParkingSpots` contains some null serialized entries.

The current code cleans them during `Awake()`.

This is not an immediate blocker, but the list should eventually be cleaned in the scene.

Trigger-exit leak

`SafetyTriggerDetector` tracks individual Collider instances.

If an object has multiple Colliders, or a Collider is disabled or destroyed while inside the Safety zone, the corresponding exit event may not occur.

Consequence

The car may remain permanently stopped.

The detector should eventually track logical entities rather than raw Collider instances, or include cleanup logic.

---

7. Recommended Fix Sequence

Phase 1 — Layer and pedestrian physics foundation

1. Select one authoritative `DynamicUser` layer.
2. Remove the duplicate layer naming.
3. Assign all pedestrians, vehicles, and relevant obstacles consistently.
4. Update the Safety detector LayerMask.
5. Add appropriate pedestrian Colliders.
6. Verify the Physics collision matrix.
7. Test basic trigger enter and exit behavior.

Phase 2 — Safety architecture cleanup

1. Decide whether the embedded or standalone Safety prefab is authoritative.
2. Remove duplicate runtime adapters.
3. Remove reflection-based dependency injection.
4. assign audio and light references explicitly.
5. Repair or remove broken serialized events.
6. establish one speed-control owner.
7. implement smooth braking and reliable resume.
8. fix multi-Collider and destroyed-object handling.

Phase 3 — Internal-screen reference architecture

1. Replace the global `GameObject.Find`.
2. bind the screen controller and hazard overlay to the correct runtime vehicle.
3. verify inactive overlays can be activated.
4. preserve and restore the previous screen state after the Safety event.

Phase 4 — Recall and ETA architecture

1. make `ValetGuidanceSystem` the authoritative recall entry point.
2. remove duplicated card-tap ETA requests.
3. update the `E` fallback to use the same recall flow.
4. resolve the correct car-specific `CarETAOrchestrator`.
5. inject the authoritative `ParkingTrafficProvider`.
6. verify that traffic and Safety delays affect the ETA.

Phase 5 — Figma UI integration

After the system connections are stable:

1. import and configure all Figma assets;
2. complete the Totem UI;
3. complete all billboard UI states;
4. complete the internal-screen route states;
5. configure aspect ratios, materials, transparency, filtering, and compression;

6. connect UI states to real project events.

Phase 6 — Integrated validation

Test the complete scenario:

1. player-car spawn;
2. gate entry;
3. drop-off;
4. pedestrian Safety interaction;
5. smooth stop;
6. warning audio and lights;
7. internal hazard overlay;
8. vehicle resume;
9. auto-parking;
10. Totem recall;
11. correct vehicle route;
12. correct ETA and traffic delay;
13. billboard update;
14. pick-up;
15. final exit.

---

8. Current Conclusion

The merged project contains the required base systems, but the current problems are mainly integration and ownership issues rather than missing high-level features.

The immediate priority should not be the visual implementation of the Figma assets.

The project should first establish:

- one authoritative pedestrian layer;
- valid physics interactions;
- one authoritative Safety architecture;
- one authoritative recall path;
- correct runtime references for the instantiated player car.

Once these foundations are stable, the ETA, Internal Screen, and Safety UI work can be completed without repeatedly reworking their connections.